

A New Encryption Algorithm for Secure Aviation Communications

Moatsum Alawida^{1b}, *Member, IEEE*

Abstract—As an integral application within the aerospace and electronic engineering systems, the aviation system relies heavily on secure communication protocols to safeguard sensitive information exchanged during travel and operations. However, many existing communication systems lack adequate security, leaving them vulnerable to adversarial attacks and data exfiltration. While effective in many security protocols, classical encryption algorithms are susceptible to various attacks, including statistical, differential, and side-channel attacks. To address these vulnerabilities, a new cryptographic encryption algorithm is proposed based on the extended Feistel network structure and binary tree structure. This algorithm enhances security measures by segmenting plaintext into blocks of 1024 bits, further divided into left and right halves across four distinct levels. Encryption begins at Level 64, employing a summation-based algorithm and XOR operations to ensure diffusion and confusion properties. The parallel implementation of encryption enhances processing speed while deriving subkeys from a sensitive secret key enhances security against single-bit changes. Experimental assessments and security analyses demonstrate the robustness of the proposed cipher against various attacks. The proposed cipher offers high-quality encryption capabilities, making it an ideal candidate for inclusion in the secure communication protocols of aviation systems.

Index Terms—Adversarial attacks, aviation communication system, cryptographic encryption algorithm, extended Feistel network, secure communication protocols.

I. INTRODUCTION

IN AVIATION environments, the continuous and seamless exchange of information between aircraft and air traffic controllers (ATCs) is crucial for maintaining safe and efficient operations. These communications, which also extend to pilots and ground controllers, involve the transmission of a wide range of data, including flight plans, operational details, cockpit communications, and maintenance records [1]. Much of this data contain sensitive and mission-critical information, such as aircraft performance metrics, positional data, weather updates, and maintenance logs [2]. Securing the transmission and storage of such data is vital to protect aviation systems from unauthorized access and malicious attacks. Historically, some air-to-ground communications relied on unencrypted

radio channels, leaving sensitive information vulnerable to interception and exploitation [2], [3]. Furthermore, the increasing use of cloud storage for data analysis and operational tasks has introduced additional security challenges. Consequently, robust and secure communication channels are essential to safeguard critical aviation data from potential adversaries [4].

The exchange of information between aircraft and ATC towers plays a central role in enabling real-time monitoring and coordination of flights. Aircraft identification, along with frequently transmitted data such as position and altitude, is shared via secondary surveillance radar (SSR), which allows ATC to track aircraft movements and provide timely instructions to ensure safety and efficiency [2], [5]. Additionally, sensitive messages, including flight plans, weather updates, and maintenance data, are transmitted through specialized data link communication technologies. Among these, the future air navigation system (FANS) and the aeronautical telecommunications network (ATN) are widely deployed [6], [7]. However, these systems differ significantly in their approach to security. While FANS is widely used, it often relies on optional data link security protocols, making it less robust in terms of protection [2], [8]. In contrast, ATN incorporates standardized and comprehensive security measures across all levels of communication, offering a higher degree of protection against potential threats.

The organization of this article as Section II outlines secure communication in aviation. Section III introduces a new cipher tailored to the unique demands of aviation communication. Section IV presents the outcomes of experiments. Section V presents the discussion critically examining the experimental results in the context of the proposed cipher's efficacy and potential vulnerabilities. Finally, Section VI concludes this article.

A. Related Work

In aviation environments, secure communication protocols are essential to achieve critical security goals: confidentiality, integrity, and authenticity [9]. Confidentiality ensures that exchanged data remain secure and cannot be accessed by unauthorized parties. Integrity ensures that any tampering or alteration of exchanged or stored data is detected. In addition, authentication ensures that access is granted only to authorized parties with proof of identity. These security goals can be achieved through the implementation of a single secure protocol, establishing secure channels between senders and receivers. Confidentiality is typically achieved through

Received 27 January 2025; revised 22 June 2025; accepted 18 September 2025. Date of publication 30 September 2025; date of current version 20 November 2025. This work was supported by Abu Dhabi University under Grant 19300887. This article was recommended by Associate Editor P. Fierens.

The author is with the Department of Computer Sciences and Information Technology, Abu Dhabi University, Abu Dhabi, United Arab Emirates (e-mail: moatsum.alawida@adu.ac.ae).

Color versions of one or more figures in this letter are available at <https://doi.org/10.1109/TSMC.2025.3613057>.

Digital Object Identifier 10.1109/TSMC.2025.3613057

encryption algorithms, which convert plaintext into unreadable ciphertext, making it difficult for unauthorized parties to interpret the data. Encryption is performed at the sender's end, while decryption occurs at the receiver's end to convert the ciphertext back into plaintext [10].

Some articles discussed secure aviation communication between pilots and ATCs. Smith et al. [11] studied and examined the Aircraft Communications Addressing and Reporting System (ACARS) encryption and discovered that the encryption could be broken on the fly using simple methods such as frequency analysis. Most of the encryption used is a mono-alphabetic substitution cipher for data transmission in wireless avionic systems. Smailes et al. [12] introduced another method to attack ATC communications, particularly the voice content. The controller pilot data link communications (CPDLC) system is crucial for sharing voice instructions in all voice communication calls. Due to large volumes of voice communications, congestion issues arise. The authors suggested using text-based communications and attacking the communication system based on software-defined radio (SDR). Man-in-the-middle attacks were used to intercept and manipulate communication undetected. Vulnerabilities were discovered, and countermeasures were proposed to address the loopholes. They made some lightweight alterations to the protocol by using logical checks and public key exchange to enhance security.

In CPDLC, voice calls are the standard communication method between ATC and pilots. Very high-frequency (VHF) bands are used for line-of-sight communication, while high-frequency (HF) bands are utilized for long-distance communication. Strohmeier [13] demonstrated the current lack of security within the automatic dependent surveillance-broadcast (ADS-B) system, highlighting that unauthenticated and unencrypted data links can be exploited in CPDLC communications. Another study illustrated the use of secure communication through asymmetric encryption to generate a securely shared symmetric session key [14]. The proposed solution is a standard approach and does not take the specific characteristics of aviation communication into consideration.

Olive [15] proposed a new secure model to enhance the security of air traffic navigation while reducing security overhead. Furthermore, another study addressed the security issues in CPDLC by introducing elliptic curve cryptography (ECC) as a combined asymmetric and symmetric encryption model for secure communication [16]. Gurtov et al. [17] studied the characteristics and parameters of CPDLC technology and proposed a set of possible countermeasures for cybersecurity threats. They introduced a solution based on ECC and proposed a new protocol, the host identity protocol (HIP). In their paper, they demonstrated that CPDLC communications are vulnerable to various cyberattacks, including eavesdropping, jamming, flooding, injection, alteration, and masquerading.

The ADS-B system plays a critical role in communication between ATC and pilots, enabling real-time tracking and monitoring of aircraft movements. As an integral component of the next-generation air transportation system, ADS-B automatically transmits and broadcasts navigation data about aircraft positions and trajectories. By enhancing

communication in regions where radar coverage is unavailable, ADS-B significantly improves the efficiency and accuracy of aircraft tracking. Despite these benefits, the system has notable security vulnerabilities, leaving it susceptible to exploitation [18].

To address these security concerns, researchers have proposed countermeasures such as spread spectrum (SS) techniques and symmetric encryption [18]. SS protects broadcast communication by spreading the signal frequency from a narrow band across a wider band, making it harder for malicious actors to intercept or disrupt the signal [19]. Two prominent SS techniques applied to ADS-B communication are direct sequence SS (DSSS) and frequency hop SS (FHSS). While these methods enhance broadcast security, they introduce challenges, including increased message sizes and the necessity of presharing identical key sequences for multiple communication sessions.

In symmetric encryption, Wu et al. [20] proposed three general methods to secure ADS-B communications using cryptographic symmetric schemes: message authentication code (MAC), format-preserving encryption, and selective field encryption. Each of these methods, however, faces specific challenges in their implementation. For instance, appending a certain bit as a MAC alters the size of the ADS-B messages, requiring global acceptance of this modification, which can be difficult to achieve within the current aviation infrastructure. Similarly, selective or format-preserving encryption partially converts the message into ciphertext without meaning, leaving other parts in plaintext. This approach violates the existing clear message exchange protocols, as the current system expects the entire message to remain human-readable and interpretable in its transmitted format [18], [21].

Several studies have explored three techniques involving classical symmetric and asymmetric encryption algorithms, focusing on key pair management and global distribution challenges. Yang et al. [22] examined the limitations of existing systems and proposed new solutions, including format-preserving encryption, vector homomorphic encryption, time-efficient stream loss-tolerant authentication (TESLA), and location privacy management. These methods aim to address critical issues like authentication and the key-sharing problem, which were previously unresolved.

To tackle these challenges, several secure schemes have been proposed [23], [24]. Identity-based encryption (IBE) has been introduced as a promising solution for authentication, where the identities of communicating parties are used to generate private keys, reducing the reliance on certificates in conventional key exchange protocols. In this approach, each aircraft is assigned a unique identity, which is utilized to facilitate both authentication and symmetric encryption [23].

Furthermore, Boyen [24] proposed a hybrid approach where a random key is selected for data encryption. The messages are encrypted using a block cipher, while IBE is employed to securely share the encryption keys. This combination enhances both the security and efficiency of ADS-B communications.

Other researchers have also contributed to addressing the key exchange protocol challenges [25], [26]. Samuelson et al. [25] proposed a signature-based method, where the signature is

divided into multiple parts to protect the identity of the aircraft. The key pairs are generated at the airport and used throughout the flight. This approach eliminates the need to send the full signature with every message; instead, only a portion of the signature is transmitted, which is then reassembled at a later stage. This method reduces communication overhead while ensuring the security of ADS-B transmissions.

Other secure ADS-B protocols have been proposed [27], [28], [29], such as the lightweight encryption scheme introduced by Yang et al. [27], which uses format-preserving encryption to encrypt only the aircraft address, leaving other messages clear. Zeng [28] also proposed another solution to enable sequential searches within encrypted data while maintaining confidentiality and integrity. Additionally, hash functions and asymmetric encryption were utilized in other approaches to ensure message integrity and authentication [29]. While these efforts provide schemes to secure ADS-B communications by addressing confidentiality, integrity, and authentication, none of the proposed solutions focus on introducing a new block cipher tailored specifically for aviation communication that can offer high security with low overhead and global applicability.

B. Problem Statement

The encryption algorithms can be symmetric and asymmetric key ciphers. A symmetric key cipher uses one secret key for both sides, while an asymmetric key cipher uses two different keys for both sides. A symmetric cipher is used more for providing confidential messages, whereas an asymmetric key cipher is used for exchanging keys between two parties [10], [30]. The most popular symmetric key ciphers are advanced encryption standard (AES) and data encryption standard (DES), both block ciphers. AES is widely regarded as more secure than DES. AES can encrypt plaintext using a secret key in three different key lengths [31], [32].

Traditional ciphers can be vulnerable to side-channel attacks [33], [34], [35], including timing analysis, power analysis, electromagnetic analysis, cache-based attacks, fault injection, and differential power analysis (DPA) [36], [37]. Among these, electromagnetic analysis and cache-based attacks are considered particularly risky because they can intercept information before and during encryption and decryption operations [38]. Electromagnetic analysis captures emissions from the CPU during encryption, revealing sensitive data about secret keys or computational processes. Cache-based attacks monitor CPU cache behavior to extract information about cryptographic operations [39], analyzing access patterns to deduce sensitive details [40]. In common traditional ciphers, predefined tables like S-boxes and P-boxes are loaded into RAM and cache before encryption begins. These tables contain highly sensitive data and are accessed rapidly during the encryption processes [41], [42].

To mitigate the risks posed by these types of attacks, various methods can be used [43], including randomizing access patterns and implementing secure cache management strategies [44], [45]. In addition, noise can be generated simultaneously with cryptographic operations to prevent information

leakage during processing. Secure cache management can involve applying encryption algorithms to cache contents or utilizing hardware-based security features [46], [47]. However, these methods may incur higher costs, especially when exchanging large data over open channels, as they require additional processing resources [40], [48].

Traditional ciphers have been used to secure channels and protect exchanged data in aviation communication. However, the demands of aviation communication require robust ciphers that can withstand various forms of attack, including side-channel and statistical attacks. AES and DES rely on predefined tables such as P-boxes or S-boxes [49]. Vulnerabilities arise when these tables are accessible in RAM or cache, or CPU before the encryption algorithm starts, allowing attackers to obtain this information directly. They can achieve this by using sensors to detect chips and reconstruct data. Consequently, robust ciphers should rely solely on the secret key and the algorithm itself rather than depending on other predefined data like S-boxes. The main limitations faced by classical ciphers can be summarized as follows.

- 1) Predefined tables such as S-boxes and P-boxes are often vulnerable to attacks since their contents can be exposed through side-channel or cache-based attacks, compromising the overall security [49].
- 2) Side-channel attacks, including timing analysis, power analysis, and electromagnetic leaks, can reveal sensitive cryptographic information by observing external factors such as energy consumption or emitted radiation [50].
- 3) Traditional ciphers, including widely used algorithms like AES and DES, fail to meet the evolving security demands of critical applications such as aviation, where resilience to various advanced attacks is essential [49].
- 4) The reliance on predefined data, such as S-boxes and P-boxes, weakens the resilience of classical ciphers by introducing potential weaknesses that can be exploited if attackers gain access to this information [41].

C. Contributions

This article introduces a novel cipher designed to address the primary weaknesses found in traditional ciphers. The proposed cipher utilizes a secret key to encrypt data blocks across multiple rounds. The secret key undergoes expansion in each round to match the size of the levels. Unlike traditional ciphers, the proposed cipher does not rely on predefined tables, and key expansion can be implemented in parallel during block generation. Additionally, encrypted blocks can be processed in parallel across different CPU cores, preventing all data from being accessed within a single core. The block data, comprising 1024 bits, is partitioned into four levels, with each level subdivided into equal-length left and right parts. The encryption process involves using the left side of each level to encrypt the corresponding right side. This methodology is applied across all four levels, with each level utilizing a different key length. The proposed cipher demonstrates the capability to encrypt files of various sizes while achieving robust security and statistical results. The main contributions can be summarized as follows.

- 1) A novel hierarchical cipher architecture is proposed, combining a multilevel Feistel network with a binary tree-based division mechanism. This structure enables scalable encryption with enhanced diffusion and confusion properties, making it suitable for securing large data blocks (1024 bit) commonly required in aviation communication systems.
- 2) A lightweight, summation-based key scheduling algorithm is introduced, generating dynamic subkeys across encryption levels. This approach improves resistance against cryptanalytic attacks, including chosen plaintext, differential, and algebraic attacks, by ensuring high key sensitivity and avalanche effect.
- 3) The cipher incorporates block-wise parallel encryption using a mask-driven operation at each level, significantly reducing execution time while maintaining robust security. The design supports efficient implementation on general-purpose processors and embedded platforms alike.
- 4) A comprehensive evaluation is conducted by implementing the proposed cipher alongside standard Feistel-based ciphers (e.g., DES, Blowfish, highly integrated lightweight 16-round block cipher (HIGHT), and Camellia) on a unified hardware platform. Results demonstrate competitive performance in terms of execution time, energy consumption, and memory usage.

II. FRAMEWORK FOR SECURE COMMUNICATION IN AVIATION

Secure communication between all stakeholders in the aviation system is vital for ensuring privacy, safety, and efficient air travel operations. The framework covers the secure communication protocol between aircraft, airports, and ATC. Communication should first be established by creating a session key and then using cryptographic algorithms for a secure channel. In this article, encryption is the main operation in the secure communication protocol, and other cryptographic operations such as ECC can be used for key exchange and secure hash algorithm (SHA3) for integrity, along with common techniques like firewalls and intrusion detection systems, can be employed.

The framework comprises three key components, each representing a stakeholder in aviation communication.

- 1) *Aircraft communication systems*: These systems are equipped with communication devices enabling aircraft to transmit and receive messages. They utilize cryptographic algorithms to ensure security goals.
- 2) *Airport communication infrastructure*: This infrastructure facilitates message exchange between aircraft and ATC, as well as among aircraft. It includes ground-based systems, antennas, and other network infrastructure.
- 3) *ATC towers*: Serving as central hubs, ATC towers manage all air traffic communication between aircraft and ground-based systems. They play a crucial role in ensuring the efficiency and safety of air travel operations.

Fig. 1 shows the three key components of the framework and the exchange of messages between them. Another critical component is the encryption and authentication mechanism.

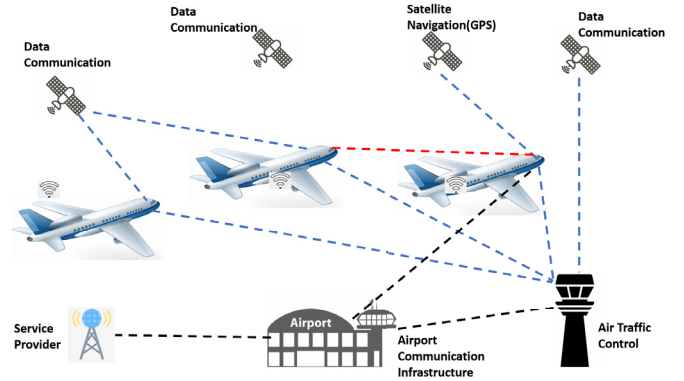


Fig. 1. Framework for message exchange in the aviation system.

The encryption algorithm requires a secret key that should be the same between the two parties. Thus, the authentication mechanism ensures that two parties are authorized to contact and exchange messages securely. Encryption is an important mechanism as it protects sensitive information related to flight operations. Therefore, this article focuses on proposing a new block cipher for exchanging information in a robust, secure, and safe manner.

III. PROPOSED CIPHER

This section outlines the development of a more flexible and secure version of the Feistel structure by reducing the number of rounds and increasing the size of the secret key with plaintext. The new cipher adopts the Feistel structure network and enhances it for improved security and increased plaintext size. The plaintext is divided into blocks, each containing 1024 bits. In level 512, each block is further divided into left and right sides, each with a size of 512 bits. Level 256 splits each side into left and right with sizes of 256 bits. Level 128 divides into left and right sides, each with a size of 128 bits. In the final level, level 64, each side from level 128 is further divided into right and left sides, each with a size of 64 bits.

Encryption begins at level 64; the left side is used to encrypt the right side of the plaintexts, ensuring the correct implementation of diffusion and confusion features. This process is applied to all sides in level 64. Then, the encrypted right and nonencrypted left sides in level 64 are swapped to create left/right sides for level 128. The process is repeated for levels 128, 256, and 512 until the encrypted plaintext is generated. The proposed cipher has one round to all levels. The secret key, initially 128 bits, is expanded in level 128 to match the size of the left and right sides. This expansion process is repeated for levels 256 and 512. The phases of the cipher can be explained in detail as follows.

A. Key Expansion

This phase is very important for confusion prosperity and to increase the key sensitivity in the proposed cipher. The secret key (k) is 128 bits, and in level 64, k divides into two sides K_L^4 and K_R^4 . In level 64, there is no expanding process; the process is a division. In level 128, 128 bits can be used for one side, and another side needs another 128 bits. Thus, the

expansion process starts from level 128. Two halves of the secret key K_L^4 and K_R^4 are concatenated to be K_R^3 . The second key K_L^3 in Level 128 is computed and extracted from K_R^3 . The steps to generate K_L^3 are as follows.

- 1) The summation-based algorithm is used; the summation of K_L^4 and K_R^4 is represented as sum1 and sum2, respectively. The summation equations are given as

$$\begin{aligned} \text{sum1} &= \text{sum1} + K_L^4(j) \times j \\ \text{sum2} &= \text{sum2} + K_R^4(j) \times j \end{aligned} \quad (1)$$

where j refers to the positions of the bits in the two half keys ($j = 1-64$).

- 2) Generate a new value enc_i using the equation

$$\text{enc}_i \equiv (\text{sum1} \times 2^8 + \text{sum2} \times 2^8 + \text{sum3}) \mod 2^{16} \quad (2)$$

where sum3 indicates the sum values for enc_{i-1} , where the first value is zero.

- 3) Convert enc_i to 16 bits and repeat until K_L^3 is generated, requiring eight rounds ($i = 1-8$).

To generate K_R^2 and K_L^2 , where K_R^2 comprises K_R^3 and K_L^3 together (concatenate process). K_L^2 requires applying the same steps as the generation of K_L^3 with the difference being in the number of rounds and positions. Here, $j = 1-128$ and $i = 1-16$ are utilized to generate K_L^2 . To generate K_R^1 and K_L^1 , the same procedure as in the previous steps is followed. K_R^1 is generated by concatenating K_R^2 and K_L^2 together. Similarly, K_L^1 is generated using the same steps as for generating K_L^3 , with the difference lying in the number of rounds and positions. Here, $j = 1-256$ and $i = 1-32$ are utilized to generate K_L^1 . Algorithm 1 outlines the steps for generating and expanding the subkeys.

B. Encryption Process

In the encryption process, four levels are used: Level 64, Level 128, Level 256, and Level 512. Each level is capable of encryption before progressing to higher levels as shown in Fig. 2. Level 64 comprises eight left sides and eight right sides, Level 128 consists of four of each side, Level 256 contains two of each side, and Level 512 possesses one left side and one right side. The encryption process begins at Level 64, where the right side is encrypted followed by the left side. This process continues at higher levels, with the same encryption procedure repeated using different sizes.

C. Level 64 Encryption

The eight right sides can be encrypted in parallel. Fig. 3 shows the encryption processes of the right sides. The encryption process proceeds as follows.

- 1) XOR R_i with K_R^4 to generate ER_i .
- 2) Calculate the sum of L_i using the equation

$$\text{sum}_L = \text{sum}_L + L_i^j \times j \quad (3)$$

where j denotes the bitstream positions in L_i .

Algorithm 1 Key Generation Algorithm

Data: Secret key, K 128 bits
Result: Subkeys, $K_R^4, K_L^4, K_R^3, K_L^3, K_R^2, K_L^2, K_R^1, K_L^1$

- 1 // Level 64: Divide the secret key into two halves
- 2 K_L^4 = first 64 bits of K ;
- 3 K_R^4 = last 64 bits of K ;
- 4 // Level 128: Concatenate halves to generate K_R^3
- 5 $K_R^3 = K_R^4 \parallel K_L^4$;
- 6 // Generate K_L^3
- 7 $\text{sum1} = 0$;
- 8 $\text{sum2} = 0$;
- 9 **for** $j = 1$ to 64 **do**
- 10 $\text{sum1} = \text{sum1} + K_L^4(j) \times j$;
- 11 $\text{sum2} = \text{sum2} + K_R^4(j) \times j$;
- 12 $\text{sum3} = 0$;
- 13 **for** $i = 1$ to 8 **do**
- 14 $\text{enc}_i = (\text{sum1} \times 2^8 + \text{sum2} \times 2^8 + \text{sum3}) \mod 2^{16}$;
- 15 $K_L^3((i-1) \times 16 + 1 : 16 \times i) =$
 convert enc_i to 16 bits;
- 16 **for** $s = 1$ to 16 **do**
- 17 $\text{sum3} = \text{sum3} + \text{enc}_i(s) \times s$;
- 18 // Level 256: Concatenate K_R^3 and K_L^3 to generate K_R^2
- 19 $K_R^2 = K_R^3 \parallel K_L^3$;
- 20 // Generate K_L^2 (similar steps as Level 128 but with different ranges)
- 21 // Level 512: Concatenate K_R^2 and K_L^2 to generate K_R^1
- 22 $K_R^1 = K_R^2 \parallel K_L^2$;
- 23 // Generate K_L^1 (similar steps as Level 128 but with different ranges)

- 3) Divide ER_i into **four blocks** of size 16 bits. To encrypt a target block, calculate the sum for the three remaining blocks without the targeted one

$$\text{sum}_R = \text{sum}_R + ER_i^j \times j \quad (4)$$

where j refers to the bitstream positions in ER_i , and the bitstream for the target block is zero in the summation. The summation of the plaintext values with their index positions increases the dependency within the encryption process. Even small changes to the plaintext produce significantly different summation results.

- 4) Use the equation below to find the mask blocks (Mask)

$$\text{Mask} = (\text{sum}_R \oplus (\text{sum}_R \gg 7) \oplus \text{sum}_L) \& 0xFFFF \quad (5)$$

where $\oplus$ denotes the bitwise XOR operation, $\gg$ represents a right shift (or rotation) by 7 bits, and $\&$ is the bitwise AND operator.

- 5) XOR Mask with the target block in ER_i^k .
- 6) Repeat steps 3 and 4 for all four blocks to encrypt the entire ER_i^k , $k = 1, 2, 3, 4$.
- 7) XOR L_i with K_L^4 to generate EL_i , and XOR ER_i with K_R^4 to generate new ER_i .

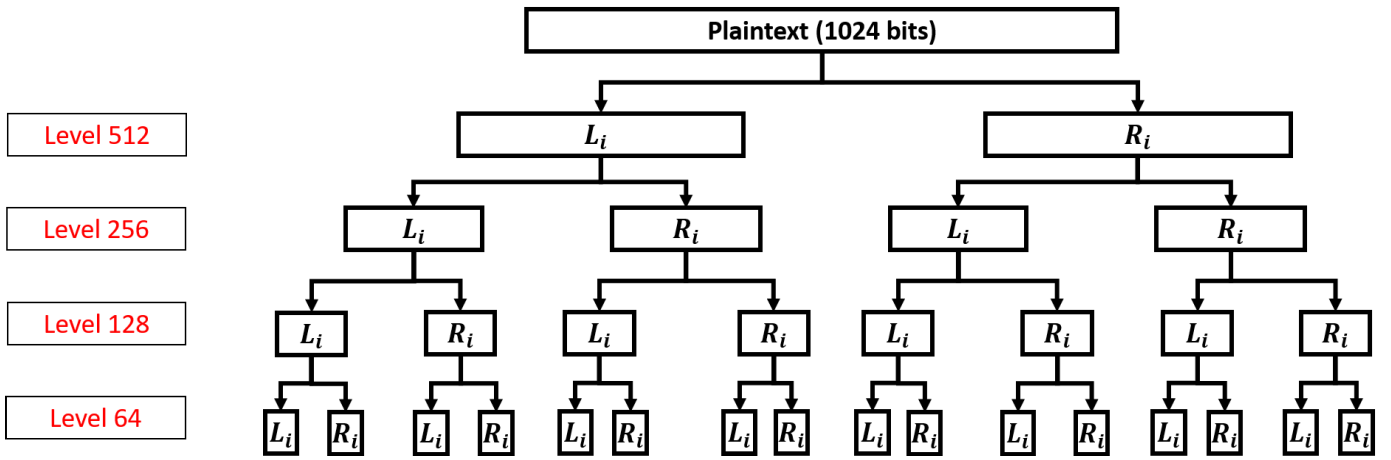


Fig. 2. Plaintext divided into four levels.

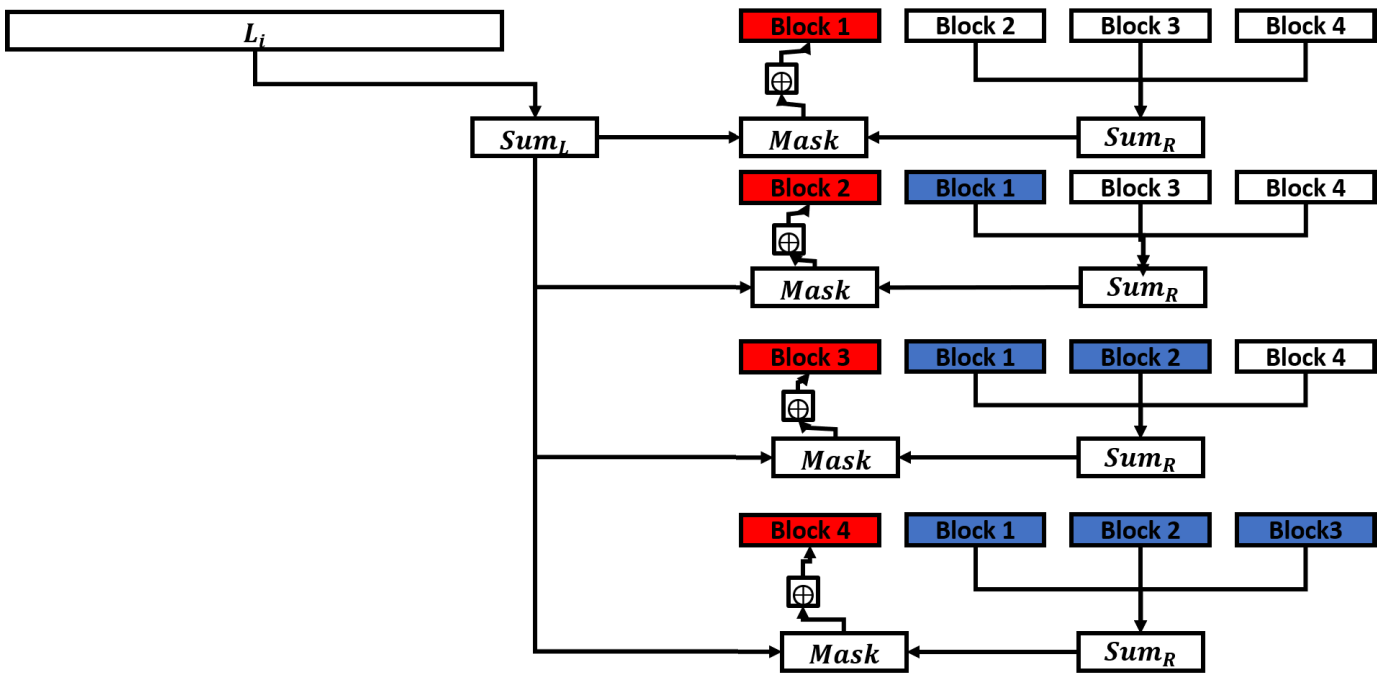


Fig. 3. Encryption process on the right side of level 64 (the blue block is the encrypted block, and the red block is the target block).

- 8) Assign R_i with EL_i and L_i with ER_i , then repeat steps 1–7 once again. After this process, both the left and right sides are encrypted.

The *mask* plays a crucial role in the proposed cipher, as it establishes a link between the left and right sides, as well as among the elements within the right side. This interconnection contributes to achieving the diffusion property in the encrypted data. Increasing the size of the mask beyond 16 bits reduces the number of blocks and weakens the linkage between individual bits. A 16-bit mask offers a balanced trade-off between computational speed and the security level of the proposed algorithm. In Level 512, the number of bits is distributed between 0s and 1s. When calculating the total for the left side, only the 1s contribute to the result, while the 0s have no effect. Since the number of 1s is approximately half of 512, we estimate 256 ones. The sum of their indices is calculated as $256 \times (255)/2 = 32\,640$, which is less than 2^{16} .

At this level, all right sides are encrypted using the corresponding left sides. Subsequently, each pair of left and right sides is combined to form a single side—either left or right—to construct Level 128. A swapping operation between the left and right sides is performed to achieve the desired permutation. Algorithm 2 illustrates the encryption process of the right sides R_i at Level 64.

D. Level 128 Encryption

The encryption process involves four left and four right sides, which can be executed in parallel mode. The encryption steps are the same as Level 64, with the only difference being the number of blocks in ER_i . Here, **eight blocks** are used instead of **four blocks** in Level 64. This change is because dividing 128 bits by the size of one block (16 bits) equals **eight blocks**. Two parts of the secret keys, K_R^3 and K_L^3 , are

Algorithm 2 Encryption Process of Right Sides R_i of Level 64

Data: Plaintext P (1024 bits), Secret key K (128 bits)
Result: Ciphertext ER (64 bits)

```

1 // Level 64 encryption
2 for  $i = 1$  to 8 do
3    $ER_i = R_i \oplus K_R^4$ ;
4    $sum_L = 0$ ;
5   for  $j = 1$  to 64 do
6      $sum_L = sum_L + L_i^j \times j$ ;
7   for  $k = 1$  to 4 do
8     Divide  $ER_i$  into four blocks of size 16 bits;
9      $sum_R = 0$ ;
10    for  $j = 1$  to 64 do
11      if  $j$  is in the bitstream positions of the target
12        block then
13        Continue;
14       $sum_R = sum_R + ER_i^j \times j$ ;
15       $Mask_k = (sum_R \oplus (sum_R \gg 7) \oplus sum_L) \& 0xFFFF$ ;
16       $ER_i^k = ER_i^k \oplus Mask_k$ ;
17    $EL_i = L_i \oplus K_L^4$ ;
18    $ER_i = ER_i \oplus K_R^4$ ;

```

Algorithm 3 Encryption Process of Right Sides R_i of Level 128

Data: Intermediate data from Level 64 (512 bits), Secret key K (128 bits)
Result: Encrypted right sides ER (128 bits)

```

1 // Level 128 encryption
2 for  $i = 1$  to 4 do
3    $ER_i = R_i \oplus K_R^3$ ;
4    $sum_L = 0$ ;
5   for  $j = 1$  to 128 do
6      $sum_L = sum_L + L_i^j \times j$ ;
7   for  $k = 1$  to 8 do
8     Divide  $ER_i$  into eight blocks of size 16 bits;
9      $sum_R = 0$ ;
10    for  $j = 1$  to 128 do
11      if  $j$  is in the bitstream positions of the target
12        block then
13        Continue;
14       $sum_R = sum_R + ER_i^j \times j$ ;
15       $Mask_k = (sum_R \oplus (sum_R \gg 7) \oplus sum_L) \& 0xFFFF$ ;
16       $ER_i^k = ER_i^k \oplus Mask_k$ ;
17    $EL_i = L_i \oplus K_L^3$ ;
18    $ER_i = ER_i \oplus K_R^3$ ;

```

utilized. At this level, all right sides are encrypted using the corresponding left sides, and then each pair of left and right sides is swapped and combined to form a single side, either left or right, to construct Level 256. Algorithm 3 shows the encryption process of right sides R_i of level 128.

E. Level 256 Encryption

The encryption process involves two left and two right sides, which can be executed in parallel mode. The encryption steps

Algorithm 4 Encryption Process of Right Sides R_i of Level 256

Data: Intermediate data from Level 128 (512 bits), Secret key K (128 bits)
Result: Encrypted right sides ER (256 bits)

```

1 // Level 256 encryption
2 for  $i = 1$  to 2 do
3    $ER_i = R_i \oplus K_R^2$ ;
4    $sum_L = 0$ ;
5   for  $j = 1$  to 256 do
6      $sum_L = sum_L + L_i^j \times j$ ;
7   for  $k = 1$  to 16 do
8     Divide  $ER_i$  into 16 blocks of size 16 bits;
9      $sum_R = 0$ ;
10    for  $j = 1$  to 256 do
11      if  $j$  is in the bitstream positions of the target
12        block then
13        Continue;
14       $sum_R = sum_R + ER_i^j \times j$ ;
15       $Mask_k = (sum_R \oplus (sum_R \gg 7) \oplus sum_L) \& 0xFFFF$ ;
16       $ER_i^k = ER_i^k \oplus Mask_k$ ;
17    $EL_i = L_i \oplus K_L^2$ ;
18    $ER_i = ER_i \oplus K_R^2$ ;

```

are the same as Level 64, with the only difference being the number of blocks in ER_i . Here, **16 blocks** are used instead of **four blocks** in Level 64. This change is because dividing 256 bits by the size of one block (16 bits) equals **16 blocks**. Two parts of the secret keys, K_R^2 and K_L^2 , are utilized. At this level, all right sides are encrypted using the corresponding left sides, and then each pair of left and right sides is swapped and combined to form a single side, either left or right, to construct Level 512. Algorithm 4 shows the encryption process of right sides R_i of level 256.

F. Level 512 Encryption

The encryption process involves one left side and one right side, following the same steps as in Level 64, with the only difference being the number of blocks in ER_i . Here, **32 blocks** are used instead of **four blocks** in Level 64. This change is because dividing 512 bits by the size of one block (16 bits) equals **32 blocks**. Two parts of the secret keys, K_R^1 and K_L^1 , are utilized. At this level, encryption is performed on the right side. Subsequently, the left and right sides are swapped and combined to form a single side, resulting in a final ciphertext of 1024 bits from the plaintext. One of the blocks from Level 64 remains unencrypted by XORing with the mask value throughout the three levels (64, 128, and 256). Therefore, in Level 512, Block 5 is swapped with Block 3 to ensure complete encryption, as illustrated in Fig. 4. Eventually, all plaintext elements are encrypted using the mask and secret key. Fig. 4 presents the encrypted blocks at each level. In the final level, all blocks are fully encrypted.

IV. EXPERIMENTAL RESULTS

In this section, the security aspects of the proposed cipher are analyzed, with a focus on statistical properties and

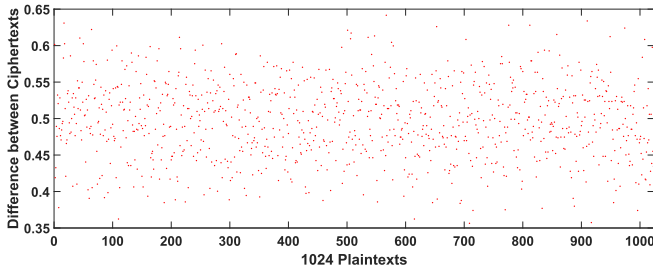


Fig. 6. Absolute difference between ciphertexts for 1024 plaintexts with one bit difference.

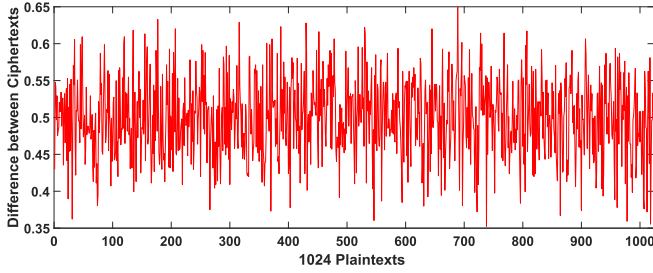


Fig. 7. Absolute difference between ciphertexts for 1024 plaintext zero values.

- 3) Compute the absolute difference between corresponding bits in $EP1$ and $EP2$.
- 4) Sum up all the absolute differences.
- 5) Divide the sum by the length of the plaintext (1024 bits in this case) to get the absolute difference between the two ciphertexts.

The formula is as follows:

$$C(i) = \frac{1}{1024} \sum_{j=1}^{1024} |EP1_j - EP2_j| \quad (6)$$

where the following holds.

- 1) $C(i)$ is the difference between the two ciphertexts after changing one bit in the plaintext.
- 2) $EP1_j$ is the j th bit of the first ciphertext.
- 3) $EP2_j$ is the j th bit of the second ciphertext.

Fig. 6 shows the absolute difference between the ciphertexts for 1024 plaintexts. 1024 positions in the original plaintext were changed to generate 1024 ciphertexts; each time, one bit was flipped from the original plaintext. Fig. 7 shows the same absolute difference with plaintext consisting of 1024 zero values. Each time, one zero was flipped to one. One can observe that the absolute difference values are near 50%, which implies high sensitivity to individual bit changes highlighting the cryptographic strength of the encryption algorithm used.

Second, each of the 128 bits in the secret key is flipped one at a time, and the corresponding ciphertexts are generated. Afterward, the difference between the two ciphertexts, resulting from the change of a single bit in the secret key, is calculated. The plaintext remains the same for all 128 iterations. Fig. 8 illustrates the difference between 128 ciphertexts for each secret key. The absolute difference is observed to be

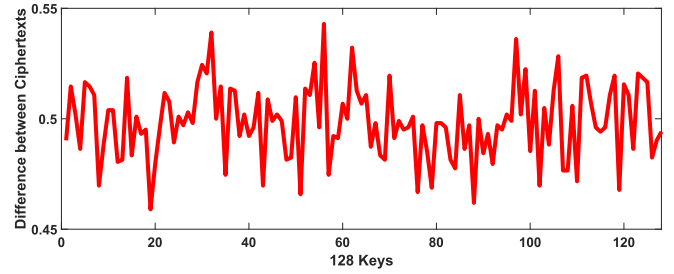


Fig. 8. Difference between 128 ciphertexts after changing one bit in the secret key.

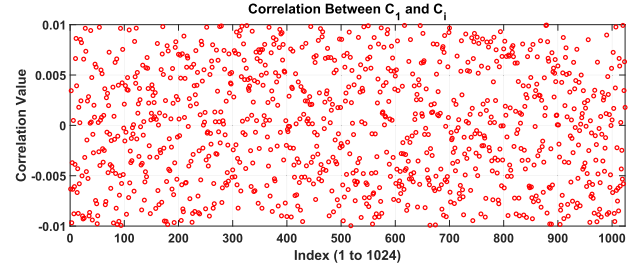


Fig. 9. Correlation values of 1024 ciphertexts.

near 50%, which indicates proximity to the theoretical value for an excellent cipher with strong confusion properties.

C. Correlation and Shannon Entropy Tests

The correlation is very important for the diffusion property of a cipher scheme, as it can be calculated between adjacent elements in plaintexts and ciphertexts. The correlation in plaintext is expected to have a high value, while the correlation in ciphertext should ideally have a low value to reflect the breaking of the relationship between adjacent elements. In this test, a plaintext P_0 with zero values is generated and encrypted to obtain ciphertext C_0 . C_0 is then divided into two parts, and the correlation between these two parts is calculated. The correlation is 0.001. In another test, 1024 plaintexts P_i by toggling one bit in each from the original plaintext P are generated. These 1024 plaintexts are encrypted to obtain ciphertexts C_i , and then calculate the correlation between the ciphertext C_1 and each of the ciphertexts C_i . Fig. 9 shows the correlation between C_1 and C_i . The proposed cipher exhibits correlation values near zero, implying no correlation between adjacent elements.

Shannon entropy is the average unpredictability of data. As ciphertext is considered random values, entropy represents a random source with maximum values of ciphertext. High entropy means high randomness and uncertainty. In this test, 8 is the theoretical value of entropy (perfect randomness). Shannon entropy can be calculated as

$$H(X) = - \sum_{i=0}^{n-1} P(x_i) \log_2 P(x_i) \quad (7)$$

where x_i is the i th symbol (e.g., byte value from 0 to 255), $P(x_i)$ is the probability of occurrence of symbol x_i , and n is the total number of possible symbols (for 8-bit data, $n = 256$).

TABLE II
COMPARISON OF ENTROPY VALUES FOR CIPHERTEXT
GENERATED BY DIFFERENT CIPHERS

Plaintext Size	Proposed Cipher	AES	DES	PRESENT
1 MB	7.9983	7.9996	7.9981	7.9984
5 MB	7.9991	7.9991	7.9975	7.9987
10 MB	7.9994	7.9991	7.9989	7.9981

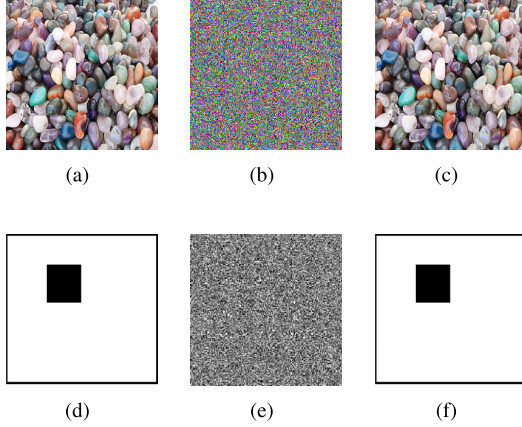


Fig. 10. Plain image, encrypted, and decrypted images for two test images.

TABLE III
CORRELATION AND SHANNON ENTROPY COMPARISON
FOR TWO TEST IMAGES

Image	Cipher	Correlation	Entropy
3*Image 1	AES	0.0042	7.9932
	DES	0.0354	7.9750
	Proposed	0.0025	7.9986
3*Image 2	AES	0.0028	7.9901
	DES	0.0287	7.9612
	Proposed	0.0019	7.9921

In practice, for a ciphertext file with N total bytes, let f_i be the frequency of byte i . Then,

$$P(x_i) = \frac{f_i}{N}$$

$$H(X) = -\sum_{i=0}^{255} \left(\frac{f_i}{N} \cdot \log_2 \frac{f_i}{N} \right). \quad (8)$$

The resulting entropy value ranges from: $0 \leq H(X) \leq 8$, with 8 being the maximum entropy for 8-bit data. Table II shows the entropy values of the proposed cipher compared with other ciphers. The proposed cipher achieved high randomness with values close to the ideal entropy, showing comparable performance to the others. This high entropy is due to the encryption of 16 bits in each level and the permutation of different block sizes across the levels.

Correlation and Shannon entropy become more evident when tested on large datasets such as digital images. Therefore, Fig. 10 displays the encrypted versions of the plain images along with their corresponding decrypted images. Two statistical tests were applied to both images, and the results are summarized in Table III. As shown, the proposed cipher effectively encrypts various types of plaintext data while maintaining strong statistical properties within a short time frame.

D. Speed Analysis

The speed of the cryptosystem is a very important criterion for cryptographic algorithms. The speed of the cryptosystem depends on many factors, such as the number of rounds, the number of mathematical operations in the cipher, the size of the plaintext block, the size of the whole plaintext, the type of processor, and memory size. In the proposed cipher, the number of rounds is 15, with eight rounds in level 64. Each part has two rounds, and eight rounds can be run in parallel mode. Similarly, in level 128, four rounds can be run in parallel mode, and in level 256, two rounds can be run in parallel mode. Thus, a total of four rounds are performed in one thread.

The mathematical operations in the proposed cipher include summation, multiplication, rotation, and logical XOR operations. While these operations contribute to increased security, they can also impact the algorithm's speed. However, the multiplication operation in this cipher is limited to multiplying a number by 1 or 0, which requires minimal computational time and does not significantly affect performance. The size of the block is 1024 bits, larger than classical ciphers such as DES and AES, which helps increase the size of the encrypted files. Other factors vary from one device to another. Therefore, in this test, DES and AES were conducted on the same machine. MATLAB 2021A was used to implement the proposed cipher along with the comparative algorithms. The implementation was carried out on a standard PC with the following specifications: flash memory: variable (GB/TB SSD/HDD), voltage: 110–240 V ac, clock speed: 2800–2900+ MHz, processor: Intel¹ Core² i7, and RAM: 16.0 GB.

The same plaintext size is used for all three ciphers to ensure a more accurate comparison. In each encryption, a single plaintext size is considered. AES takes 128 bits in one block, and it has ten rounds to generate ciphertext. Thus, 1024 bits need 80 rounds to generate ciphertext with a size of 1024 bits. DES takes 64 bits in one block and 16 rounds to generate ciphertext. Thus, it requires 256 rounds to encrypt 1024 bits. The proposed cipher executed on the parallel mode for four levels. While AES and DES rely heavily on predefined S-boxes and permutation tables combined with XOR operations, they avoid using multiplication and summation during the encryption process itself. While AES does use addition (summation) within its MixColumns step, these calculations involve predefined constants, allowing them to be precomputed beforehand. This eliminates the need for dynamic calculations during runtime encryption.

In contrast, the proposed cipher utilizes multiplication operations to generate the mask for the 16-bit data across four levels. While a single multiplication might take around six clock cycles, the cumulative effect of these runtime calculations contributes to the longer execution time compared to AES and DES. Table IV presents the comparison of encryption/decryption times for different plaintext sizes across three ciphers. It is evident from Table IV that the proposed cipher

¹Registered trademark.

²Trademarked.

TABLE IV
COMPARISON OF ENCRYPTION SPEED FOR DIFFERENT PLAINTEXT SIZES (SECONDS)

Plaintext Size	Proposed Cipher	Proposed Cipher (Parallel Mode)	AES	DES	Decryption
1 MB	0.1421	0.0272	0.1350	0.0280	0.0856
5 MB	1.6451	0.3421	0.5879	0.4514	0.6532
10 MB	3.356	1.0102	1.242	1.021	1.9856

TABLE V
COMPARISON OF OTHER CLASSICAL CIPHERS

Cipher	Time in seconds (1 MB)	Diffusion	Confusion	Key Space	Block Size / Rounds
AES	0.1350	50.12%	50.18%	Large (128-256 bits)	128 bits / 10-14 rounds
DES	0.0250	49.7%	50.7%	Small (56 bits)	64 bits / 16 rounds
3DES	0.1855	50.17%	50.19%	Limited (112 bits)	64 bits / 48 rounds
Blowfish	0.1595	50.35%	50.38%	Large (32-448 bits)	64 bits / 20 rounds
RC6	0.3014	49.65%	51.08%	Large (128, 192, or 256 bits)	128 bits / 16 rounds
RC5	0.1878	49.08%	49.18%	Large (0 to 2040 bits)	(32, 64 or 128 bits) / (1-255) rounds
PRESENT	0.0775	49.25%	49.58%	(80 and 128 bits)	64 bits / 31 rounds
The proposed cipher	0.0272	50.13%	50.04%	Large (128 bits)	1024 bits / 15 rounds

demonstrates longer encryption times compared to DES and AES when operating in nonparallel mode.

In parallel mode, MATLAB 2021A supports parallel processing through its parallel computing toolbox, and Intel Core i7 typically has eight physical cores. Encryption rounds can be divided among multiple threads to ensure faster execution. A `parpool` is initialized to activate parallel workers, and a `parfor` loop is used to distribute the encryption of the left and right sides across available threads. This process is repeated for all encryption rounds to complete the operation efficiently. The execution can be distributed across multiple threads as follows.

- 1) At **Level 64**, each thread handles one round of the left and right sides, utilizing a total of eight threads.
- 2) At **Level 128**, the workload is divided among four threads.
- 3) At **Level 256**, the process is executed using two threads.
- 4) Finally, at **Level 512**, the execution is performed using a single thread.

Table IV also presents a comparison of encryption times. It can be observed that the proposed cipher achieves shorter encryption times compared to DES and AES when operating in parallel mode. However, some challenges can face the parallel computing of the proposed cipher, including thread management, synchronization, and scalability. In Levels 64 and 128, MATLAB `parfor` uses multiple threads to handle the blocks, increasing the utilization of the CPU's multicore capabilities. Buffer handling and sequential swapping are used to maintain data consistency across blocks and achieve synchronization. This has an effect on the overhead caused by large intermediate variables. Another scalability challenge was observed when encrypting Levels 256 and 512, where the computational overhead slightly increased compared to Levels 64 and 128.

E. Comparison With Other Ciphers

This study compares the key cryptographic properties of the proposed cipher with those of AES, DES, triple data encryption standard (3DES), Blowfish, RC5, RC6, and present encryption standard (PRESENT). Critical attributes such as

diffusion, confusion, and the correlation to the key space are investigated. The objective is to demonstrate that the new algorithm is effective, innovative, and suitable for practical use, especially in aviation communication systems that require enhanced security and efficiency.

Table V presents a comparative analysis of the seven ciphers based on key criteria. The proposed cipher exhibits performance comparable to the four classical ciphers in terms of security metrics. This is due to the careful integration of diffusion and confusion properties across four levels, along with the strategic swapping between the left and right sides of the data. The proposed cipher has features that overcome those of traditional ciphers. The secret key is 128 bits and can be increased to another size using the same algorithm key expansion. It does not rely on predefined tables to encrypt plaintexts, making it robust for use in aviation environments and any applications that prioritize security regardless of implementation time.

Feistel structure is used in many ciphers in both classical and lightweight methods. The proposed cipher is built upon a combination of binary tree structure and Feistel network, along with a summation-based method and XOR operations. The swapping between sides at each level contributes to achieving the permutation feature. Comparison with other Feistel-based encryption algorithms is important to show what is new in the proposed cipher and to highlight its main achievements. Table VI displays four well-known ciphers that use the Feistel structure in their algorithm design. The execution time was measured based on encrypting a 1024-bit plaintext block using each cipher. The proposed cipher can encrypt in 0.0652 ms, making it faster than the other ciphers. The energy estimate is calculated by multiplying the device's power consumption (in watts) by the time taken to perform the encryption (in seconds), as shown in the equation below

$$\text{Energy (Joules)} = \text{Power (Watts)} \times \text{Time (Seconds)}. \quad (9)$$

A low energy value is generally more desirable than a high value. As we can see from Table VI, the proposed cipher has an average energy consumption comparable to other ciphers.

TABLE VI
PERFORMANCE COMPARISON WITH FEISTEL-BASED CIPHERS (TESTED ON STANDARD PC: INTEL CORE i7, 16 GB RAM)

Cipher	structure	Block Size (bits)	Key Size (bits)	Rounds	Exec. Time (ms)	Energy (mJ)	Memory Usage (MB)
DES	Feistel	64	56 (effective)	16	0.1451	42.0	11.52
Blowfish	Feistel	64	32–448	16	0.5124	40.1	10.2
HIGHT	Unbalanced Feistel	64	128	32	0.0986	38.8	11.0
Camellia	Feistel-like (SPN)	128	128, 192, 256	18/24/32	0.0985	38.5	12.3
Proposed Cipher	Tree-Feistel	1024	128	Multi-level (15)	0.0652	39.5	11.3

Another important metric is memory usage, which refers to the total RAM consumed during the encryption phase. Low memory usage is preferable, especially for devices with limited RAM. The proposed cipher's memory usage is also comparable to other ciphers. However, increasing the plaintext size may lead to higher hardware resource consumption. Several other differences between the proposed cipher and existing Feistel-based ciphers are listed below.

- 1) The proposed cipher uses four hierarchical levels, whereas other Feistel-based ciphers typically rely on a single-level Feistel structure with fixed round functions.
- 2) A binary tree structure is applied in the proposed cipher, with recursive division across levels and upward merging, while traditional Feistel-based ciphers generally perform only recursive division of plaintext.
- 3) A masking technique is used in the proposed cipher to achieve diffusion, compared to other Feistel-based ciphers that use only round-based mixing methods.
- 4) At each level, the right sides are encrypted in parallel blocks, while other Feistel-based ciphers do not implement parallelism in their design.
- 5) The block size scales with the level in the proposed cipher, while other ciphers typically use a fixed block size throughout the entire encryption process.
- 6) The secret key is scheduled in three different sizes for each level, unlike conventional designs that use a single subkey size across all rounds.

F. Security Proofs

The secret key is first analyzed to demonstrate its security when one bit is changed. Let $K \leftarrow \{0, 1\}^k$ denote the key space, where k is the key length. Let $M \leftarrow \{0, 1\}^P$ represent the message space with a size of 1024 bits. E is the proposed cipher, and $C \leftarrow \{0, 1\}^P$ is the ciphertext with the same size as M . The adversary A aims to construct two distinct secret keys and the same message M , such that $M_1 = M_2$, and $K_1 \neq K_2$. The difference between the two keys is one bit, that is, $|K_1| - |K_2| = 1$. $E(K_1, M_1)$ and $E(K_2, M_1)$ are implemented and generate two distinct ciphertexts (C_1, C_2) , where $C_1 \neq C_2$.

The difference between two distinct ciphertexts (C_1, C_2) is near to 50%.

Consider the following levels in the encryption process where K_R^4 and K_L^4 are used to construct K_R^3 . In level 128, the difference between K_1^3 and K_2^3 remains one bit, that is, $K_1^3 - K_2^3 = 1$. However, the difference between K_1^3 and K_2^3 is greater than 50%, because Algorithm 1 creates 16 bits in each iteration based on 128 bits. Therefore, the total difference in level 64 is $50\% + 1$.

In level 2, the difference between K_1^2 and K_2^2 is $50\% + 1$, and the difference between K_1^2 and K_2^2 is greater than 50%. Thus, the total difference is $50\% + 1 + 50\%$.

In the last level, the difference between K_1^1 and K_2^1 is $50\% + 1 + 50\%$, and the difference between K_1^1 and K_2^1 is greater than 50%. Therefore, the total difference is $50\% + 1 + 50\% + 50\%$.

Finally, considering 896 bits in K_1 and K_2 , the difference is more than 50%.

Therefore, the difference between two distinct ciphertexts (C_1, C_2) is near to 50%.

A aims to construct two distinct messages M_1 and M_2 with the same secret key K , such that $M_1 \neq M_2$. The difference between the two messages is one bit, that is, $|M_1| - |M_2| = 1$. $E(K, M_1)$ and $E(K, M_2)$ are implemented and generate two distinct ciphertexts, $C_1 \neq C_2$.

The difference between two distinct ciphertexts (C_1, C_2) is near to 50%.

At level 64, one out of eight parts is completely changed, i.e., $L_{c_1} \neq L_{c_2}$, while for the other parts $R_{c_1} = R_{c_2}$, resulting in the same ciphertext. The proportion is calculated as $\left| \frac{50}{8} - 100 \times \frac{7}{8} \right|$, which equals 18.75%. At level 128, the proportion is $\left| \frac{50}{4} - 100 \times \frac{3}{4} \right| = 37.5\%$. At level 256, the proportion is $\left| \frac{50}{2} - 100 \times \frac{1}{2} \right| = 50\%$. At level 512, the proportion is $\left| \frac{50}{1} - 100 \times \frac{1}{1} \right| = 50\%$.

G. Resistance to Cryptanalytic Attacks

Some cryptographic attacks pose significant security challenges, and ciphers must be designed to be resistant to them, particularly differential, linear, or algebraic attacks. Ciphers that rely on substitution and permutation operations are vulnerable to these attacks due to their dependence on approximating the cipher's behavior. These attacks rely heavily on the specific secret key in use, meaning that any approximation of the cipher's structure remains valid only for that particular key, rendering it ineffective for other keys. For example, a differential attack becomes possible when the difference between differential probabilities of various data streams is propagated. This vulnerability is particularly evident in systems that utilize S-boxes or modulo addition operations, as these differential properties must be understood before the encryption process begins. In the proposed cipher, the final step involves XORing the intermediary encrypted data with subkeys across four levels to enhance protection against cipher behavior analysis. Furthermore, the inclusion of dynamic operations such as XORing, summation, and positional bitstream calculations strengthens the cipher's resistance to differential attacks. These

operations ensure that the encryption process remains robust and unpredictable, effectively mitigating vulnerabilities.

In the proposed cipher, nonlinear operations are derived from two key XOR processes: the XOR of the plaintext or intermediate ciphertext with the mask at all levels and the XOR with subkeys across four levels. While XOR is inherently a linear operation, the mask values are generated through complex summation, and positional bitstream calculations, all of which involve multiple nonlinear components. Similarly, although XORing with subkeys is also a linear operation, the subkeys themselves are produced from intricate nonlinear components. To even begin estimating the differential properties of these operations, an attacker would require knowledge of the key values or subkeys, achievable only through exhaustive key search or brute force attacks. In other words, differential cryptanalysis provides no computational advantage over brute force attacks. Due to the inherent duality between differential and linear cryptanalysis, this argument applies equally to the latter, further reinforcing the cipher's robustness against such attacks.

When it comes to algebraic attacks, nonlinear operations like S-boxes are modeled as a system of algebraic equations with more equations than unknowns. The proposed cipher employs a dynamic substitution method, using a 16-bit mask to replace each plaintext word in a unique way depending on its position. This makes it extremely difficult for an attacker to create a set of equations that accurately represent the substitution process. Even if they could, they should have to create a new set of equations for every different plaintext length. Given the random and dynamic nature of the proposed substitution, the proposed cipher is assumed to be highly resistant to algebraic attacks.

Chosen plaintext attacks (CPAs) are a form of cryptanalysis in which an opponent selects plaintexts with fixed patterns to try to extract relationships between the components of the cipher. In the proposed cipher, the deep dependency on both key variation across levels and the internal diffusion mechanism presents a significant challenge to such attacks. Three main techniques: sum-based key scheduling, block-wise XOR, and mask operations—ensure that the ciphertext exhibits a strong avalanche effect. Additionally, other design factors contribute to disrupting the plaintext–ciphertext correlation. For example, the encryption process depends on each block's index position, left/right relationships, and accumulated sums, making it very difficult to discover any patterns or predictable mappings between plaintexts and their corresponding ciphertexts.

V. DISCUSSION

The proposed cipher incorporates several security strengths worth highlighting. By employing the Feistel network structure, a new cipher was developed with a novel approach to achieving diffusion and confusion properties. Additionally, several techniques were integrated into the cipher to further enhance its security, including the following.

- 1) *Key expansion*: Subkeys are derived from the secret key using a summation-based algorithm. This involves summing the elements of the secret key to generate a

new subkey, which is then utilized to generate additional subkeys. Unlike other key scheduling methods where subkeys remain constant, the proposed approach ensures that each subkey is dynamically generated at each level of encryption, thereby enhancing security. Additionally, incorporating element positions into the summation process adds an extra layer of complexity, further increasing the confusion property of the cipher. This process can be performed during the encryption itself, eliminating the need to execute it beforehand or store it in RAM or cache memory. As a result, the key expansion becomes more resistant to side-channel and cache-based attacks.

- 2) *Substitution and permutation*: In the proposed cipher, substitution is performed using a summation-based algorithm and XOR operation. All intermediate encrypted elements are utilized to generate a new 16-bit mask for encrypting 16 bits of plaintext. This technique is applied at all levels, resembling nonlinear substitution, where each 16-bit block uses other left and right sides to generate a new 16 bits. The input varies between levels: in Level 16, 64 bits from the left side and 48 bits from the right side are used to generate a 16-bit mask for the right side, resulting in 112 bits used to generate 16 bits. In Level 128, 240 bits are input to generate a 16-bit mask; in Level 256, 496 bits are input to generate a 16-bit mask; and in the final Level 512, 1008 bits are used as input to generate a 16-bit mask. Therefore, this technique acts as a nonlinear substitution with varying input sizes to produce a fixed-size output. As for permutation, the swapping between left and right sides scrambles the plaintext elements without requiring additional time, and no predefined table is used.
- 3) *Diffusion and confusion*: Diffusion is achieved using a summation-based algorithm with an XOR operation. Any change in the plaintext at any position results in a ripple effect in Level 64 within the same block, which propagates to Levels 128, 256, and 512. Consequently, after encrypting 1024 bits, a single-bit change has a full effect on the entire ciphertext. Similarly, for confusion, a one-bit change in the secret key affects all blocks in Level 64, resulting in a new ciphertext at Level 64. In the subsequent levels, the ciphertexts are entirely new because both the subkeys and intermediate encrypted elements are new. These properties are attained through several factors, including the summation-based algorithm, XORing with the secret key and subkeys, and XORing with the mask (16 bits).

A. Cache/Side Channel Attack

The proposed cipher does not rely on predefined tables such as P-boxes or S-boxes. Additionally, no data are initialized in the cache or main memory before the encryption begins. The cipher simultaneously takes the plaintext and the secret key and can be distributed across eight cores to operate in parallel mode during Level 64. For Levels 3, 2, and 1, the key is expanded to match the size of the plaintext. The key expansion process is integrated into the encryption operations,

meaning it is performed dynamically during the encryption process itself. Consequently, there is no need to preprocess and store all subkeys in RAM or cache memory prior to the encryption. This design ensures that the proposed cipher offers the advantage of protecting against side-channel and cache-based attacks.

B. Aviation Communication System

The proposed cipher is well-suited for aviation communication systems due to its robust security, making it difficult to attack through traditional methods such as statistical or side-channel attacks. In addition, it can accommodate block sizes larger than those of other traditional ciphers. Since most of the data exchanged between airplanes and ATC, or airline operations, are not the largest files but rather contain highly sensitive flight data, security is of utmost importance. Therefore, the proposed encryption employs complex mathematical operations to generate robustly secure ciphertexts.

Some limitations in the proposed cipher may lead to vulnerabilities to attack and cryptanalysis. The first one is the direct use of the secret key in level 64 without generating additional subkeys. This leaves the cipher vulnerable, as attackers can potentially extract the secret key from level 64 and then mount attacks on the entire cipher to generate plaintext. To address this vulnerability, subkeys can be generated for level 64 dynamically rather than being preprocessed, thus preventing attacks through side-channel vulnerabilities. The second limitation lies in the size of the mask, which is set at 16 bits. With the computational power of modern computers, attackers could potentially generate all possible 16-bit masks and apply them to each block across all levels. While theoretically feasible, this approach would require significant computational resources. To mitigate this type of attack, increasing the size of the mask can enhance complexity, making it more difficult for attackers to guess the mask and thus increasing overall security.

VI. CONCLUSION

In conclusion, the proposed cipher was developed based on the Feistel network structure and incorporates various techniques aimed at enhancing the security of aviation communications. The plaintext is partitioned into blocks, each with a size of 1024 bits, and further divided into left and right halves to create four levels. Encryption commences at Level 64 using a summation-based algorithm with XOR operations. The parallel implementation of encryption enhances processing speed. Subkeys are generated from a secret key that is highly sensitive to single-bit changes. Experimental results and security analyses demonstrate the proposed cipher's resilience against statistical attacks and cryptanalysis techniques, while also showcasing its ability to encrypt data within a short timeframe. Aviation communications can use the proposed cipher to generate robust ciphertexts. The proposed cipher can resist side-channel attacks without the need for additional defense strategies. Future work will explore dynamic permutation and extend the binary tree with the Feistel network structure to further enhance security and speed.

ACKNOWLEDGMENT

Sincere appreciation is also extended to the anonymous reviewers and editors for their insightful feedback and constructive suggestions, which have significantly enhanced the quality and clarity of this article.

REFERENCES

- [1] H.-H. Yang, Y.-H. Chang, and Y.-H. Chou, "Subjective measures of communication errors between pilots and air traffic controllers," *J. Air Transp. Manage.*, vol. 112, Sep. 2023, Art. no. 102461.
- [2] E. Habler, R. Bitton, and A. Shabtai, "Assessing aircraft security: A comprehensive survey and methodology for evaluation," *ACM Comput. Surveys*, vol. 56, no. 4, pp. 1–40, Apr. 2024.
- [3] X. Chen et al., "Covert communications: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 1173–1198, 2nd Quart., 2023.
- [4] A. E. Omolara, M. Alawida, and O. I. Abiodun, "Drone cybersecurity issues, solutions, trend insights and future perspectives: A survey," *Neural Comput. Appl.*, vol. 35, no. 31, pp. 23063–23101, Nov. 2023.
- [5] A. Hamissi and A. Dhraief, "A survey on the unmanned aircraft system traffic management," *ACM Comput. Surveys*, vol. 56, no. 3, pp. 1–37, Mar. 2024.
- [6] J. Zhao, Y. Zhu, K. Cai, Z. Gao, Z. Han, and L. Hanzo, "Multiple-antenna aided aeronautical communications in air-ground integrated networks: Channel estimation, reliable transmission, and multiple access," *IEEE Wireless Commun.*, vol. 31, no. 2, pp. 105–111, Apr. 2024.
- [7] A. Brizzi et al., "SATCOM for ATM: Space segment description and technology roadmap for FOC safety-critical satellites," in *Proc. IEEE Aerosp. Conf.*, Mar. 2023, pp. 1–11.
- [8] S. Khan, G. S. Gaba, A. Braeken, P. Kumar, and A. Gurtov, "AKAASH: A realizable authentication, key agreement, and secure handover approach for controller-pilot data link communications," *Int. J. Crit. Infrastructure Protection*, vol. 42, Sep. 2023, Art. no. 100619.
- [9] G. Dave, G. Choudhary, V. Sihag, I. You, and K.-K.-R. Choo, "Cyber security challenges in aviation communication, navigation, and surveillance," *Comput. Secur.*, vol. 112, Jan. 2022, Art. no. 102516.
- [10] H. K. A. Alsuwaiedi and A. M. S. Rahma, "A new modified DES algorithm based on the development of binary encryption functions," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 35, no. 8, Sep. 2023, Art. no. 101716.
- [11] M. Smith, D. Moser, M. Strohmeier, V. Lenders, and I. Martinovic, "Economy class crypto: Exploring weak cipher usage in avionic communications via acars," in *Financial Cryptography and Data Security*. Cham, Switzerland: Springer, 2017, pp. 285–301.
- [12] J. Smailes, D. Moser, M. Smith, M. Strohmeier, V. Lenders, and I. Martinovic, "You talkin' to me? Exploring practical attacks on controller pilot data link communications," in *Proc. 7th ACM Cyber-Phys. Syst. Secur. Workshop*, 2021, pp. 53–64.
- [13] M. Strohmeier. (2016). *Security in Next Generation Air Traffic Communication Networks*. [Online]. Available: <https://api.semanticscholar.org/CorpusID:4869504>
- [14] T. McParland, V. Patel, and W. J. Hughes, "Securing air-ground communications," in *Proc. 20th DASC. 20th Digit. Avionics Syst. Conf.*, vol. 2, 2001, pp. 7A7/1–7A7/9.
- [15] M. L. Olive, "Efficient datalink security in a bandwidth-limited mobile environment—an overview of the aeronautical telecommunications network (ATN) security concept," in *Proc. 20th DASC. 20th Digit. Avionics Syst. Conf.*, vol. 2, 2001, pp. 9E2/1–9E2/10.
- [16] D. Getachew, "Simulation performance results of an authentication protocol for controller-pilot data link communications," in *Proc. 24th Digit. Avionics Syst. Conf.*, vol. 1, 2005, pp. 1–B.2.
- [17] A. Gurtov, T. Polishchuk, and M. Wernberg, "Controller-pilot data link communication security," *Sensors*, vol. 18, no. 5, p. 1636, May 2018.
- [18] H. A. Khan, H. Khan, S. Ghafoor, and M. A. Khan, "A survey on security of automatic dependent surveillance-broadcast (ADS-B) protocol: Challenges, potential solutions and future directions," *IEEE Commun. Surveys Tuts.*, early access, Dec. 9, 2024, doi: 10.1109/COMST.2024.3513213.
- [19] Y. Liu, P. Ning, H. Dai, and A. Liu, "Randomized differential DSSS: Jamming-resistant wireless broadcast communication," in *Proc. IEEE INFOCOM*, Mar. 2010, pp. 1–9.

- [20] Z. Wu, T. Shang, and A. Guo, "Security issues in automatic dependent surveillance-broadcast (ADS-B): A survey," *IEEE Access*, vol. 8, pp. 122147–122167, 2020.
- [21] Q. Zhang, Z. Wang, B. Wu, and G. Gui, "A robust and practical solution to ADS-B security against Denial-of-Service attacks," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 13647–13659, Apr. 2024.
- [22] H. Yang, H. Li, and X. S. Shen, *Secure Automatic Dependent Surveillance-Broadcast Systems*. Cham, Switzerland: Springer, 2022.
- [23] J. Baek, E. Hableel, Y.-J. Byon, D. S. Wong, K. Jang, and H. Yeo, "How to protect ADS-B: Confidentiality framework and efficient realization based on staged identity-based encryption," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 3, pp. 690–700, Mar. 2017.
- [24] X. Boyen, "A tapestry of identity-based encryption: Practical frameworks compared," *Int. J. Appl. Cryptography*, vol. 1, no. 1, pp. 3–21, 2008.
- [25] K. Samuelson, E. Valovage, and D. Hall, "Enhanced ADS-B research," in *Proc. IEEE Aerosp. Conf.*, 2006, pp. 1–7.
- [26] Z. Wu, A. Guo, M. Yue, and L. Liu, "An ADS-B message authentication method based on certificateless short signature," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 56, no. 3, pp. 1742–1753, Jun. 2020.
- [27] H. Yang, M. Yao, Z. Xu, and B. Liu, "LHCSAS: A lightweight and highly-compatible solution for ADS-B security," in *Proc. IEEE Global Commun. Conf.*, Dec. 2017, pp. 1–7.
- [28] F. Zeng, "Secure ADS-B protection scheme supporting query," in *Proc. IEEE SmartWorld, Ubiquitous Intell. Comput., Adv. Trusted Comput., Scalable Comput. Commun., Internet People Smart City Innov. (Smart-World/SCALCOM/UIC/ATC/IOP/SCI)*, Oct. 2021, pp. 513–518.
- [29] T. Kacem, D. Wijesekera, P. C. G. Costa, and A. B. Barreto, "Secure ADS-B framework 'ADS-Bsec,'" in *Proc. IEEE 19th Int. Conf. Intell. Transp. Syst. (ITSC)*, Nov. 2016, pp. 2681–2686.
- [30] M. Alawida, "A novel chaos-based permutation for image encryption," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 35, no. 6, Jun. 2023, Art. no. 101595.
- [31] O. Kara, "Square impossible differential attack and security of AES in known plaintext scenario," *Cryptologia*, vol. 49, no. 2, pp. 1–25, Mar. 2025.
- [32] F. Yu, S. He, W. Yao, S. Cai, and Q. Xu, "Bursting firings in memristive Hopfield neural network with image encryption and hardware implementation," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, early access, May 7, 2025, doi: [10.1109/TCAD.2025.3567878](https://doi.org/10.1109/TCAD.2025.3567878).
- [33] W. Feng et al., "Exploiting robust quadratic polynomial hyperchaotic map and pixel fusion strategy for efficient image encryption," *Expert Syst. Appl.*, vol. 246, Jul. 2024, Art. no. 123190.
- [34] S. H. Mekala, Z. Baig, A. Anwar, and S. Zeadally, "Cybersecurity for industrial IoT (IIoT): Threats, countermeasures, challenges and future directions," *Comput. Commun.*, vol. 208, pp. 294–320, Aug. 2023.
- [35] C. Ni and S. C. Li, "Machine learning enabled industrial IoT security: Challenges, trends and solutions," *J. Ind. Inf. Integr.*, vol. 38, Mar. 2024, Art. no. 100549.
- [36] M. M. Sravani and S. Ananiah Durai, "Attacks on cryptosystems implemented via VLSI: A review," *J. Inf. Secur. Appl.*, vol. 60, Aug. 2021, Art. no. 102861.
- [37] C. Ye, S. Tan, J. Wang, L. Shi, Q. Zuo, and W. Feng, "Social image security with encryption and watermarking in hybrid domains," *Entropy*, vol. 27, no. 3, p. 276, Mar. 2025.
- [38] E. Praveen Kumar and S. Priyanka, "A comprehensive survey on hardware-assisted malware analysis and primitive techniques," *Comput. Netw.*, vol. 235, Nov. 2023, Art. no. 109967.
- [39] C. Ashokkumar, R. P. Giri, and B. Menezes, "Highly efficient algorithms for AES key retrieval in cache access attacks," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, Mar. 2016, pp. 261–275.
- [40] S. Sharma, A. Saini, and S. Chaudhury, "A survey on biometric cryptosystems and their applications," *Comput. Secur.*, vol. 134, Nov. 2023, Art. no. 103458.
- [41] C. Qin, J. Li, and P. P. C. Lee, "The design and implementation of a rekeying-aware encrypted deduplication storage system," *ACM Trans. Storage*, vol. 13, no. 1, pp. 1–30, Feb. 2017.
- [42] F. Yu, D. Su, S. He, Y. Wu, S. Zhang, and H. Yin, "Resonant tunneling diode cellular neural network with memristor coupling and its application in police forensic digital image protection," *Chin. Phys. B*, vol. 34, no. 5, Apr. 2025, Art. no. 050502.
- [43] F. Liu, H. Wu, K. Mai, and R. B. Lee, "Newcache: Secure cache architecture thwarting cache side-channel attacks," *IEEE Micro*, vol. 36, no. 5, pp. 8–16, Sep. 2016.
- [44] J. Gonzalez-Gomez, L. Bauer, and J. Henkel, "Cache-based side-channel attack mitigation for many-core distributed systems via dynamic task migration," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2440–2450, 2023.
- [45] M. Li, K. Bu, C. Miao, and K. Ren, "TreasureCache: Hiding cache evictions against side-channel attacks," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 4574–4588, Sep. 2024.
- [46] F. Yu, S. Zhang, D. Su, Y. Wu, Y. M. Gracia, and H. Yin, "Dynamic analysis and implementation of FPGA for a new 4D fractional-order memristive Hopfield neural network," *Fractal Fractional*, vol. 9, no. 2, p. 115, Feb. 2025.
- [47] N. Khoshavi, M. Maghsoudloo, Y. Bi, W. Francois, L. Jaimes, and A. Sargolzaei, "A survey on attack vectors in stack cache memory," *Integration*, vol. 72, pp. 134–147, May 2020.
- [48] J. Zhang, C. Chen, J. Cui, and K. Li, "Timing side-channel attacks and countermeasures in CPU microarchitectures," *ACM Comput. Surveys*, vol. 56, no. 7, pp. 1–40, Jul. 2024.
- [49] C. Ashokkumar, B. Roy, M. B. S. Venkatesh, and B. L. Menezes, "S-Box' implementation of AES is NOT side-channel resistant," *J. Hardw. Syst. Secur.*, vol. 4, no. 2, pp. 86–97, Dec. 2019, doi: [10.1007/s41635-019-00082-w](https://doi.org/10.1007/s41635-019-00082-w).
- [50] K. D. Wesson, T. E. Humphreys, and B. L. Evans, "Can cryptography secure next generation air traffic surveillance?" *IEEE Secur. Privacy Mag.*, vol. 12, no. 6, p. 77, Oct. 2014.



Moatsum Alawida (Member, IEEE) received the Ph.D. degree in computer science with a specialization in cybersecurity (cryptography) from the School of Computer Sciences, Universiti Sains Malaysia, George Town, Malaysia, in 2020.

He is currently an Assistant Professor of Cybersecurity with Abu Dhabi University, Abu Dhabi, United Arab Emirates. With a prolific academic career, he has authored and published over 70 articles in high-impact factor journals. Notably, in the years 2021–2024, he achieved recognition among the top 2% of scientists worldwide. His diverse research interests encompass chaotic systems, applications of chaos, multimedia security, drone security blockchain, cybersecurity, quantum-based cryptography, and traditional cryptography methods. Beyond his research contributions, he has actively served as a Referee for esteemed journals, including IEEE, Elsevier, Springer, MDPI, and various conferences.